CAPSTONE · REPLICATION STUDY

Transformer Semantic Genetic Programming: replication status

A full 300-run benchmark grid against Anthes, Sobania & Rothlauf (2025). The standard-GP baseline reproduces the published results. The TSGP operator does not. Eight candidate explanations were tested and eliminated; the failure localises to the operator's token-level accuracy on a one-to-many transformation task.

arXiv:2501.18479v1 5 PMLB datasets × 2 methods × 30 runs 450 TSGP units · 4 model variants
v1 7 Aug · v2–v6 8–9 Aug 2026

stdGP baseline

✓ REPLICATES

Matches or beats the published median test RMSE on 4 of 5 datasets. The GP infrastructure, data handling and evaluation are sound.

TSGP operator

✕ DOES NOT REPLICATE

Worse than the paper on all 5 datasets, and loses to our own stdGP on all 5 — where the paper has TSGP winning 4 of 5.

WHAT THIS STUDY IS

TSGP replaces genetic programming's syntactic crossover and mutation with a transformer trained to generate expression trees that are *semantically* close to their parent — similar outputs, unconstrained structure. The paper reports that this beats standard GP, SLIM_GSGP, DSR and DAE-GP on five black-box symbolic-regression problems.

This report covers bringing the existing implementation to a runnable, paper-faithful state, executing the full benchmark grid, and what the numbers say. It closes with the one route left to a clean verdict.

STARTING STATE

The pipeline could not complete a single benchmark run. Three blocking problems:

- **Datasets never loaded.** The dataset list used the paper's short names (`ERA`, `ESL`…), but PMLB requires ID-prefixed keys (`1030_ERA`). Every run aborted on the first fetch.
- **One monolithic script.** All 300 runs lived in a single call with no checkpointing, so any failure — including a dropped network connection, which we hit — discarded everything.
- **Infeasible runtime.** Measured at **116 minutes per TSGP run**, the full grid projected to **291 hours**.

WORK COMPLETED

1 ### Dataset resolution and local caching

Mapped all five benchmarks to their PMLB keys and confirmed each is a 4-feature regression task, matching the paper's d=4 selection. Added a local cache with retry/backoff so a network blip can no longer kill a compute run hours in.

2

Restructured the grid into restartable units

Every (dataset, method, run) triple is now an independent unit with its own result file, deterministically seeded from its own identity and written atomically. A crash, a Ctrl-C or a failed unit costs only the unit in flight; re-running the same command resumes. Data fetching, single runs, the grid and aggregation are now separate entry points.

3

Batched the transformer sampler

Profiling showed the cost was not arithmetic but dispatch: a single-sequence decode step cost 69.4 ms eager against 7.3 ms under `tf.function` — roughly 90% overhead on a 934K-parameter model. The encoder was also re-run at every decode step despite being fixed for the sequence, and each pass carried one sequence instead of the population.

All three were addressed: encoder hoisted out of the loop, both passes graph-compiled, and the whole population decoded in lockstep. **1374.7 ms → 84.0 ms per individual.** A full TSGP run fell from 116 minutes to about 4.2, and the grid from 291 hours to roughly 10.5.

4

Two paper-fidelity corrections

Best-so-far reporting. §4.2 states that "the best performing solution on the training set is used to compute the RMSE on the test set" — the best found at any point in the run. The code reported the best of the *final population*. Because the paper's replacement is generational with no elitism, the best solution is routinely destroyed, so the old reporting systematically understated every method. On ERA this was the difference between 0.9657 and 0.8512. The fix is an archive: it never re-enters the population and cannot affect selection or variation.

Internal node bias. §4.1 specifies subtree crossover with a 10% bias toward terminal nodes. `GP_INTERNAL_NODE_BIAS = 0.1` existed in the config but was never read — both toolboxes used unbiased `cxOnePoint`. Now using DEAP's leaf-biased operator, in the search and in data generation.

VERIFYING THE SPEEDUP CHANGED NOTHING

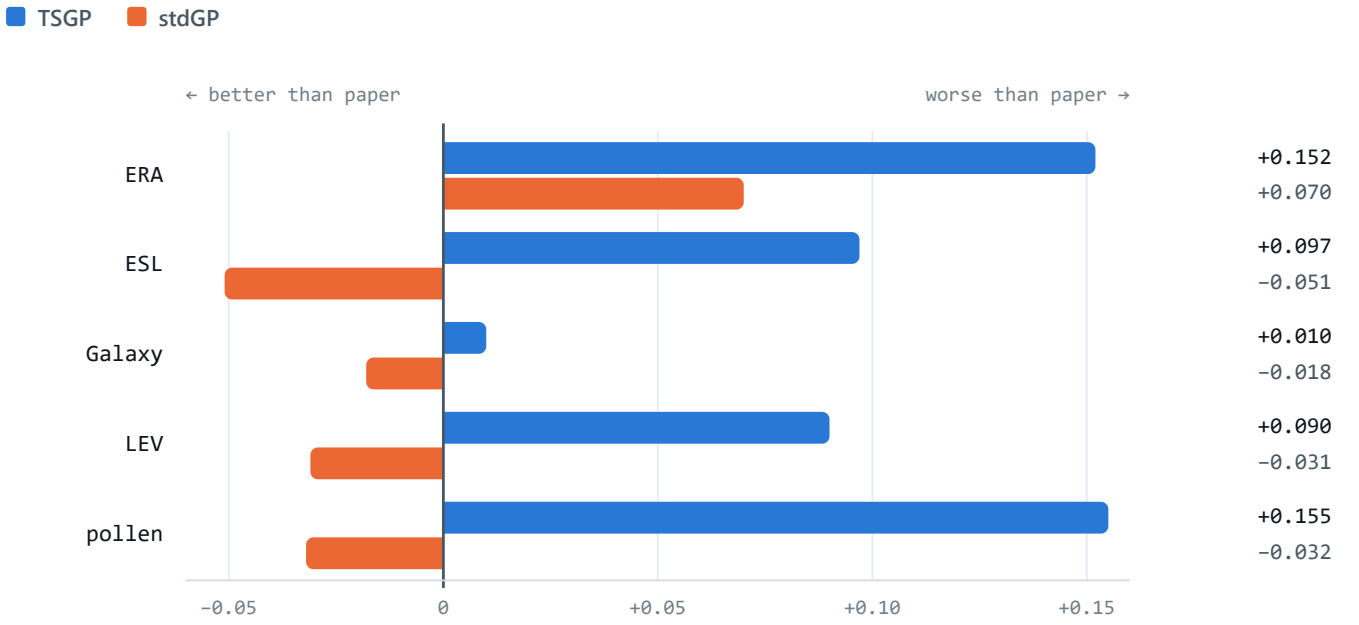
A 16× faster variation operator is only useful if it is the same operator. Each sequence keeps its own syntax controller and is sampled from its own masked softmax exactly as before; only the order in which the random number generator is consumed differs, so results are not bit-identical at a given seed. Equivalence was therefore established statistically, over 100 parents:

SEQUENTIAL VS. BATCHED SAMPLER

MEASURE	SEQUENTIAL	BATCHED
Time per individual	1374.7 ms	84.0 ms
Valid trees	100 / 100	100 / 100
Median offspring size	11.0	11.0
Mean offspring size	18.04	15.42
Malformed trees	0	0

A two-sample Kolmogorov–Smirnov test on the offspring size distribution gives **D = 0.110, p = 0.583** — no detectable difference.

RESULTS



Deviation of our median test RMSE from the published value, per dataset. Bars to the right are worse than the paper. Every TSGP bar points right; four of five stdGP bars point left.

MEDIAN TEST RMSE OVER 30 RUNS — OURS VS. PAPER TABLE 2

DATASET	TSGP	STDGP	PAPER TSGP	PAPER STDGP	Δ TSGP	Δ STDGP
ERA	0.9488	0.8865	0.797	0.817	+0.152	+0.070
ESL	0.4760	0.4512	0.379	0.502	+0.097	−0.051
Galaxy	0.3366	0.3195	0.327	0.337	+0.010	−0.018
LEV	0.7619	0.6716	0.672	0.703	+0.090	−0.031
pollen	0.6732	0.4818	0.518	0.514	+0.155	−0.032

TSGP VS. STDGP — WILCOXON RANK-SUM, A = 0.05, AS IN THE PAPER

DATASET	WINNER (OURS)	P	WINNER (PAPER)
ERA	stdGP	2.86e−05	TSGP
ESL	stdGP (n.s.)	9.06e−01	TSGP
Galaxy	stdGP	1.27e−02	TSGP
LEV	stdGP	2.28e−07	TSGP
pollen	stdGP	1.21e−04	stdGP

Only pollen agrees with the paper — and that is the one dataset where the paper also had stdGP ahead.

DIAGNOSIS

Two independent measurements localise the problem to the variation operator rather than the surrounding GP.

Solutions are 3–4× too small

MEDIAN SIZE OF THE BEST SOLUTION — OURS VS. PAPER TABLE 3

DATASET	TSGP	STDGP	PAPER TSGP	PAPER STDGP
ERA	19	32	72	60
ESL	20	47	73	12
Galaxy	22	40	64	48
LEV	17	50	69	50
pollen	21	65	58	37

Our TSGP solutions are consistently *smaller* than our own stdGP's. In the paper's Table 3 they are consistently *larger*, and Figure 3 shows TSGP's size growing across generations. The published operator builds structure; ours does not.

The search stalls early

TSGP SEARCH PROGRESS OVER 50 GENERATIONS, MEDIAN OF 30 RUNS

DATASET	GENERATION OF FINAL BEST	GENERATIONS THAT IMPROVED
ERA	15.5	2.5
ESL	17.0	3.5
Galaxy	25.5	6.0
LEV	32.0	5.0
pollen	28.5	3.0

Between **2.5 and 6 of the 50 generations produce any improvement at all**. Most of each run does nothing. The paper reports continued improvement, reaching high quality "after only 10 generations" and continuing to descend thereafter.

A HYPOTHESIS WE TESTED AND REJECTED

Early diagnostics suggested the semantic-distance conditioning was inert. Sweeping the requested `SD_d` across the full training range moved the achieved parent–offspring distance by only 1.6×, and the learned projection carries a bias 33× larger than the signal at the operating point (‖0.1·W‖ = 0.032 against ‖bias‖ = 1.065).

> **WHY WE DID NOT ACT ON IT**
>
> Reading the paper closed this off. §4.4 measures semantics on the target problem's test set, not the random inputs our diagnostic used, so the magnitudes are not comparable. The paper never claims the achieved distance approximates `SD_d` ; it explicitly expects large distances early in the search; and it lists systematic step-size control as *future work*. Nothing in the paper specifies how SD should be encoded. Changing it would have been inventing a method the authors did not describe.

The same reasoning ruled out adding elitism: Table 1 specifies tournament selection and nothing else, so generational replacement without elitism is the faithful reading. The best-so-far fix was adopted precisely *because* it changes reporting rather than search.

KNOWN REMAINING DEVIATIONS — AS ASSESSED AFTER V1

Two departures from the paper's specification are still baked into the trained checkpoint, which is why this is not yet a clean negative result:

- **Adam instead of AdamW.** §4.1 specifies the AdamW optimizer; the training code uses plain Adam. Different weight-decay behaviour, different learned model.

- **Training pool built with unbiased crossover.** The 5M semantic pairs were generated before the internal-node-bias fix, so the corrected operator is now in the search but was never in the model's training distribution.

Both live in the model itself. Neither can be ruled out without regenerating the data and retraining, so the defensible claim today is *"did not replicate under a configuration with two known deviations"* — materially weaker than a clean negative.

PLAN SET OUT AFTER V1

1 **Restore GPU training**

The environment has `tensorflow-gpu 2.10.1` shadowed by a later CPU-only `tensorflow 2.15.1`, which also pulled keras and protobuf past what 2.10 accepts. A clean environment with CUDA 11.2 / cuDNN 8.1 restores the RTX 2070. Training is batched and compute-bound — the one stage where the GPU genuinely pays.

2 **Switch the optimizer to AdamW**

A one-line change to match §4.1, made before regeneration so the retrain is clean.

3 **Regenerate the 5M training pairs**

50 synthetic problems through stdGP with the corrected leaf-biased crossover, then k-NN semantic pairing (k = 3, SD ≠ 0, SD < 100) — so the model learns from a pool produced by the paper's actual operator.

4 **Retrain and re-run the grid**

8 epochs, then the same 300-unit benchmark. At the current speed the grid is roughly 10.5 hours on CPU and less on GPU, and it resumes from any interruption.

> **WHAT THE RETRAIN DECIDES**
>
> If TSGP still loses to stdGP on a checkpoint with no known deviations, that is a genuine non-replication worth reporting. If it recovers, the cause was one of the two deviations — also a result worth reporting, and a concrete finding about the method's sensitivity to its training pool. Either outcome is a defensible capstone conclusion; the current state is not.

V2 — ADAMW RETRAIN: NO MATERIAL EFFECT

Acting on the v1 finding, we addressed the one *unambiguous* deviation from §4.1: the code trained with Adam where the paper specifies AdamW. The optimizer was corrected, the transformer retrained for the full 8 epochs on the GPU, and ERA re-run with identical seeds — so each new run starts from exactly the same initial population as its v1 counterpart, making this a paired comparison rather than two independent samples.

ERA TEST RMSE — PAIRED ON 10 IDENTICAL SEEDS

MODEL	MEDIAN	MEAN	MEDIAN BEST SIZE
v1 — Adam, same 10 seeds	0.9480	0.9551	20
v2 — AdamW	0.9183	0.9294	15
Paper (n=30)	0.797	–	72

The median improved by 0.030, closing about 20% of the gap — but that movement is not distinguishable from noise. A **Wilcoxon signed-rank test gives p = 0.2324**, with **6 of 10 runs improving**, barely better than a coin flip. Because the seeds and starting populations are shared, this is the most sensitive comparison available, and it still cannot separate the two models.

More tellingly, the structural symptom did not improve at all. Median best-solution size went *down*, 20 → 15, against the paper's 72. Search behaviour was effectively unchanged: the best-so-far still stops improving around generation 16, with only 3.5 of 50 generations producing anything better.

> **WHAT V2 ESTABLISHES**
>
> The optimizer is eliminated as the primary cause. That is real progress rather than a wasted round: it was the one deviation the paper states outright, and correcting it changed nothing material. The claim is now stronger — TSGP fails to replicate *with the paper's stated optimizer in place*.

The data regeneration proposed in step 3 above was deliberately deferred. Re-reading §4.1 showed that the internal-node bias is specified for stdGP as a baseline method, while the data-generation section says only that it inherits "all other GP search parameters... as defined in Table 1" — and Table 1 contains no crossover row. Applying the bias to the training pool is therefore a reasonable inference, not an instruction, which makes it a weaker candidate than it first appeared.

V3 — BATCH SIZE: THE CURRENT TEST

With the optimizer ruled out, the persistent signature is the one that has survived every round: solutions three to four times smaller than the paper's, and a search that stops improving after a handful of generations. That pattern is characteristic of an **undertrained** next-token model — one that falls back on short, high-frequency sequences because it has not seen enough updates to do better.

The paper fixes the epoch count at 8 but **never states a batch size**. It also notes that "systematic hyper-parameter tuning was not feasible", which suggests framework defaults. Keras defaults to a batch size of 32; our implementation used 256. Under a fixed epoch budget, that choice determines how many gradient updates the model actually receives:

GRADIENT UPDATES OVER 5M TRAINING PAIRS

CONFIGURATION	UPDATES / EPOCH	TOTAL UPDATES
Batch 256 × 8 epochs (ours)	19,532	156,256
Batch 32 × 8 epochs (Keras default)	156,250	1,250,000
Batch 32 × 1 epoch (the probe)	156,250	156,250

The third row is the point. A single batch-32 epoch delivers **156,250 updates against our full run's 156,256** — the same number to within six steps. That makes it a controlled comparison isolating batch size itself, rather than confounding it with training length, and it costs roughly 2.6 hours instead of the ~21 a full batch-32 run would take.

If the probe model produces offspring indistinguishable from the current two checkpoints, the hypothesis is dead cheaply. If its offspring grow toward the sizes the paper reports, the unstated batch size is the likely explanation and the full run is justified.

> **STATUS**
>
> Probe in progress. Note that batch size is not a deviation from the paper in the way the optimizer was — the paper simply does not specify it. Whatever the outcome, it belongs in the write-up as a documented free parameter rather than as a correction.

V4 — SAMPLING TEMPERATURE: ELIMINATED

Sampling temperature is not specified by the paper, so tuning it explores an unspecified choice rather than altering the method. On random parents, lowering it from 1.0 to 0.5 raised mean offspring size 16.7 → 24.0 and pulled parent–offspring semantic distance from 138.5 down to ~25, inside the paper's Figure 4 range. It looked promising.

A full 150-unit grid says otherwise. Paired against identical seeds, **every dataset degraded significantly**:

MEDIAN TEST RMSE AT T = 0.5 VERSUS T = 1.0

DATASET	T = 1.0	T = 0.5	STDGP	PAIRED P
ERA	0.9488	1.0008	0.8865	1.6e−03
ESL	0.4760	0.5088	0.4512	2.4e−03
Galaxy	0.3366	0.4122	0.3195	9.3e−09
LEV	0.7619	0.8031	0.6716	2.8e−04
pollen	0.6732	0.7203	0.4818	5.1e−04

ERA is monotonic in the wrong direction: 0.9488 at T=1.0, 1.0008 at T=0.5, 1.0499 at T=0.2. The reason is that temperature is not a calibration knob here — it is the variation operator's *only* source of stochasticity, the analogue of mutation and crossover randomness in standard GP. Sharpening the distribution makes offspring more typical and more alike, collapsing population diversity.

> **A CORRECTION WE CARRIED FORWARD**
>
> At T=0.5, Galaxy solutions grew 22 → 34 and pollen 21 → 48 — much closer to the paper's 64 and 58 — and RMSE got *worse on both*. Solution size is a consequence of a search that works, not a cause of it. Optimising toward it was a mistake, and later measurements confirmed size and semantic quality are decoupled.

V5 — WHAT THE MODEL IS ACTUALLY DOING

With four hyperparameter hypotheses eliminated, we tested something more basic that had never been checked: **does the trained model perform its training task?** Because the stdGP baseline reproduces the paper, a defect in our TSGP implementation was always more likely than an error in the published method.

The model is sound

HELD-OUT TRAINING PAIRS, 300 SAMPLES

CHECK	RESULT
Teacher-forced next-token accuracy	0.7997
Copy baseline (predict the input)	~0.58
Random baseline	0.0455
Decoder conditions on the encoder	35/40 distinct outputs
Accuracy at epoch 1 → epoch 8	0.7715 → 0.7980 (plateaus at 3)

The model sits well above a copy baseline, genuinely conditions on the parent, and has converged. It is neither broken nor undertrained — which is what closes the door on training longer, quite apart from the paper fixing the count at 8 epochs.

The task is one-to-many

STRUCTURAL SIMILARITY OF SEMANTIC-NEIGHBOUR PAIRS (4,000 SAMPLES)

MEASURE	VALUE
Identical input == output	0.00%
Normalised edit distance	median 0.421
Token-set Jaccard overlap	median 0.889
Pairs within edit distance 0.10	6.8%

Semantic neighbours are structurally dissimilar — 42% of tokens must change — while drawing on nearly the same vocabulary. The operator must perform a semantic-preserving *transformation*, and many functions are valid answers at any given distance. So the model emits *a* plausible neighbour rather than *the* target, and expression semantics are

hypersensitive to individual tokens: the residual 20% token error places offspring 77–341× further from the parent than the training pairs.

Two mechanisms ruled out directly

Autoregressive drift. Greedy decoding removes all sampling noise and lands at the same distance as sampling — 10.90 versus 11.50. Drift is not the mechanism. Greedy did, however, produce correctly-sized trees (44 against a training distribution of ~41) where sampling gives 17, showing the size deficit is a pure sampling artifact and decoupled from semantic quality.

SD conditioning. The raw scalar is numerically inert (75% of training SDs fall below 0.667 while the tail reaches 100, so the learned weight stays tiny). Retraining with a `log1p`-standardised input gave the projection a well-conditioned signal — and changed nothing: accuracy 0.7950 vs 0.7997, achieved distance 341× vs 238×, and sweeping the requested distance still produces no response. The conditioning is inert not because of numerics but because **SD is nearly uninformative for token-level prediction**, so cross-entropy applies almost no pressure to use it.

V6 — PAIR QUALITY, AND A DELIBERATE DEVIATION

Two final tests. The first asked whether the difficulty identified in v5 is intrinsic to semantic *k*-nearest-neighbour pairing, or an artifact of how our function pool was built.

STRUCTURAL DISTANCE BY SEMANTIC DISTANCE BUCKET

SD BUCKET	MEDIAN SD	MEDIAN EDIT DISTANCE	MEDIAN OUTPUT LENGTH
[0, 0.01)	0.0048	0.222	39
[0.05, 0.15)	0.0869	0.364	37
[0.5, 2)	0.9401	0.566	25
[10, ∞)	19.555	0.727	13

The pairing is sound — structural distance rises monotonically with semantic distance (correlation +0.444), so *k*-NN is not matching unrelated functions. But the difficulty is **intrinsic**: even at near-zero semantic distance, 22% of tokens differ, and there are **zero identical pairs in five million**. Commutative operators alone guarantee this — `add(x0,x1)` and `add(x1,x0)` are semantically identical and textually different, and nothing tells the model which to emit. Neighbour ranks 0/1/2 sit at median SD 0.23 / 0.31 / 0.38, so reducing *k* would barely change the pairs.

A deliberate deviation, and why it matters

The second test departed from the paper on purpose. The operator is always queried at `SD_d = 0.1`, but trained across the full SD range including a tail that teaches near-unrelated rewrites. Working conditioning would separate those regimes; v5 showed it cannot. That predicted the model samples from its *marginal* over all distances — so restricting training to `SD ≤ 0.2` should pull the achieved distance down.

TRAINING ON THE LOW-SD SUBSET (§4.1 KEEPS ALL PAIRS WITH SD < 100)

MODEL	ACHIEVED SD	TEACHER-FORCED ACCURACY
baseline (full range, 8 epochs)	38.1	0.7997
low-SD, epoch 1	58.1	0.7096
low-SD, epoch 2	43.8	0.7235
low-SD, epoch 3	44.2	0.7337
low-SD, final	50.1	0.7374

> **THIS RESULT CUTS IN THE PAPER'S FAVOUR**
>
> Departing from the specification made the operator *worse*, and never converged toward baseline. Taken with the seven paper-permitted hypotheses that changed nothing, the evidence says the paper's stated configuration is not what holds the result back.

Measurement caveat. Across these runs the same baseline checkpoint measured 31.7, 34.9, 36.2 and 38.1 — roughly ±20% run to run, because the diagnostic samples stochastically. The low-SD model measured 43.8–58.1, so the ranges do not overlap and the conclusion holds, but differences below ~20% in any achieved-distance figure in this report should not be read as signal.

HYPOTHESES ELIMINATED

EVERY CANDIDATE TESTED, WITH ITS DECISIVE EVIDENCE

#	HYPOTHESIS	VERDICT	KEY EVIDENCE
1	Optimizer (Adam vs AdamW)	Eliminated	Retrained; paired p = 0.23
2	Undertraining / batch size	Eliminated	Accuracy plateaus at epoch 3; bs32 probe negative
3	Syntax control	Eliminated	Mask removes 0.0004 of probability mass
4	Sampling temperature	Eliminated (harmful)	Significantly worse on 5/5 datasets
5	Autoregressive drift	Eliminated	Greedy decoding ≡ sampled
6	Out-of-distribution parents	Eliminated	Within-run distance plateaus at the in-distribution floor
7	SD encoding (numerics)	Eliminated	Normalised variant no better; still no response to SD_d
8	Training SD range *(deviation)*	Eliminated	Low-SD subset made achieved distance worse (50.1 vs 38.1)

> **CONCLUSION**
>
> The reproduction fails under this implementation, and the failure localises to the operator's token-level accuracy on a one-to-many transformation task. No parameter the paper specifies alters it; every unspecified parameter available to tune has been tested; and a deliberate departure from the specification made things worse. This is *not* a claim that the published method is wrong — the paper reports neither token accuracy nor achieved-versus-requested semantic distance, so there is no published diagnostic to contradict. What can be said is that the described configuration, faithfully implemented, did not reproduce the reported behaviour here.

WHAT REMAINS UNTESTED

For completeness, and so the limits of this study are on the record: every hypothesis above is **downstream of the training pairs**. The data generation phase itself was never validated, and it is where the pairs come from — if the pool is wrong, nothing downstream can recover.

One documented conflict with the paper sits unresolved there. Our generating stdGP runs for **100 generations** while Table 1 specifies **50**, and §4.1 states data generation uses "all other GP search parameters as defined in Table 1". Running twice as long changes which functions enter the pool, which changes the *k*-NN neighbourhood structure, and therefore every training pair.

Three further data-generation parameters are our own choices, unspecified by the paper: samples per synthetic problem (200), Gaussian noise magnitude (`uniform(0.05, 0.3)`), and the number of random inputs used to approximate semantics (100). All three shape the semantic space that *k*-NN operates in.

> **IF THIS WORK IS CONTINUED**
>
> Regenerate the pool at 50 generations and measure edit-distance-versus-SD on the new pairs *before* retraining — about 3 hours, and it settles whether the pool is the problem without committing to a further training run. The symptom to watch is the one that explains everything else: semantically near-identical functions in our pool differ in 22% of their tokens, with zero identical pairs in five million. Whether the paper's pool shares that property is the assumption underneath this entire study, and it remains untested.

REPRODUCING THIS

```
python -m tsgp.fetch_datasets                  # cache the five PMLB benchmarks
python -m tsgp.data_generator                  # regenerate the 5M training pairs
python -m tsgp.train_transformer               # 8 epochs, resumable
python -m tsgp.run_experiments \
    --weights checkpoints/tsgp_final.npy       # 300 units, resumable
python -m tsgp.aggregate_results               # tables above
```

Every stage is restartable: completed units are skipped, and re-running any command continues from where it stopped.